

Analyse et modélisation
d'informations (AMINF)

2025 – 2026

SQL

Essential Query Syntax

Nicolas MARMANN

1^{re} CI1



Lycée des Arts et Métiers

Section I – Informatique et Communication

Version of the 27.04.2026

Table of Contents

TABLE OF CONTENTS.....	2
TABLE OF FIGURES	3
TABLE OF TABLES	3
I. NORMAL FORMS IN DATABASES.....	4
I.1. Benefits of using Normal Forms	4
I.2. Normal Forms (1NF – 3NF).....	4
I.2.a. First Normal Form (1NF).....	4
I.2.b. Second Normal Form (2NF).....	4
I.2.c. Third Normal Form (3NF).....	5
I.3. Overview of Normal Forms (1NF – 3NF).....	5
II. QUERIES.....	6
II.1. SELECT	6
II.2. DELETE.....	6
II.3. INSERT.....	7
II.4. UPDATE.....	7
III. JOIN TABLES	8
III.1. LEFT JOIN.....	8
III.1.a. Full LEFT JOIN.....	8
III.1.b. LEFT JOIN without intersecting B.....	8
III.2. RIGHT JOIN	9
III.2.a. Full RIGHT JOIN.....	9
III.2.b. RIGHT JOIN without intersecting A.....	9
III.3. INNER JOIN	10
III.4. Cascade Operations.....	10
IV. AGGREGATE FUNCTIONS	12
IV.1. List of built-in SQL Functions.....	12
IV.2. Detailed Function Descriptions.....	13
IV.2.a. GROUP_CONCAT().....	13
IV.2.a.i. Syntax.....	13
IV.2.a.ii. Explanation.....	13
IV.2.a.iii. Main features	13
IV.2.b. FORMAT()	14
IV.2.b.i. Syntax.....	14
IV.2.b.ii. Explanation.....	14
IV.2.b.iii. Main features	14

V. CONDITIONS AND OPERATORS	15
V.1. Numeric Comparison Operators	15
V.2. String Comparison Operators	15
V.3. Logical Operators.....	16
V.4. Null Operators	16
V.5. Other Numeric Operators	16
VI. REFERENCES	17
VI.1. Bibliography	17
VI.2. Sources of Illustrations	17

Table of Figures

Figure I.1 – Database Normalisation.....	5
Figure IV.1 – Aggregate Functions.....	13

Table of Tables

Table V.1 – Numeric Comparison Operators	15
Table V.2 – String Comparison Operators	15
Table V.3 – Logical Operators	16
Table V.4 – Null Operators.....	16
Table V.5 – Other Numeric Operators	16

I. Normal Forms in Databases

I.1. Benefits of using Normal Forms

- Reduce duplicate data and wasted storage.
- Prevent insert, update, and delete anomalies.
- Improve data consistency and integrity.
- Make the schema easier to maintain and evolve.

I.2. Normal Forms (1NF – 3NF)

I.2.a. First Normal Form (1NF)

A table is in 1NF when all of its data is stored in a clear and atomic way.

This means:

- Every cell contains one single value only, not lists or repeating attributes.
- Each record can be uniquely identified, typically by a primary key.
- All rows must store data of the same type within a column.

Purpose:

1NF eliminates duplicate groups and ensures that the structure of the table is regular and predictable.

I.2.b. Second Normal Form (2NF)

A table is in 2NF when:

- It is already in 1NF, and
- Every non-key attribute depends on the whole primary key, not just part of it.

This rule only applies when the primary key is composite (made from more than one column).

If a non-key column depends on only one part of a composite key, that dependency must be moved to a separate table.

Purpose:

2NF removes partial dependencies, reducing redundancy that appears when attributes relate only to part of the key.

I.2.c. Third Normal Form (3NF)

A table is in 3NF when:

- It is already in 2NF, and
- Non-key attributes do not depend on other non-key attributes.

In other words, there should be no transitive dependencies.

If a column depends on another non-key column (for example, “City” depending on “PostalCode”), it should be separated into another table.

Purpose:

3NF ensures that each piece of information is stored exactly where it logically belongs, improving data consistency and avoiding update anomalies.

I.3. Overview of Normal Forms (1NF – 3NF)

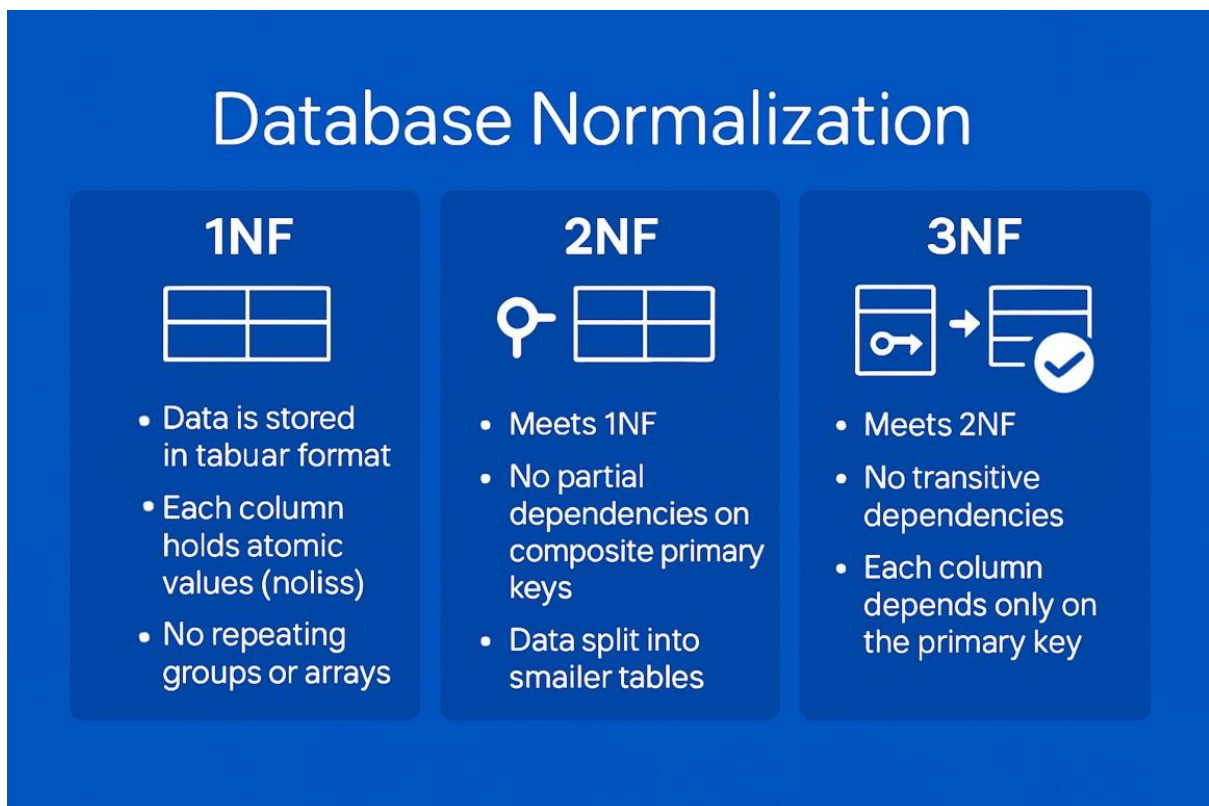


Figure I.1 – Database Normalisation

II. Queries

II.1. SELECT

Syntax

```
SELECT [DISTINCT] column_name [, column_name, ... AS other_name]
FROM table_name(s)
[WHERE expression_1
    AND expression_2
    OR expression_3]
[GROUP BY column_name, column_name, ...]
[HAVING expression_1
    AND expression_2
    OR expression_3]
[ORDER BY column_name [ASC|DESC] [, column_name, ...]]
[LIMIT row_count [OFFSET offset]] ;
```

II.2. DELETE

Syntax

```
DELETE (table_name)
FROM table_name
[WHERE expression_1
    AND expression_2
    OR expression_3] ;
```

II.3. INSERT

Syntax

```
INSERT INTO table_name
VALUES (value1, value2, ...),
      [(value1, value2, ...),
       (value1, value2, ...),
       ...] ;
```

OR

```
INSERT INTO table_name(column_name1, column_name2, ...)
VALUES (value1, value2, ...),
      [(value1, value2, ...),
       (value1, value2, ...),
       ...] ;
```

II.4. UPDATE

Syntax

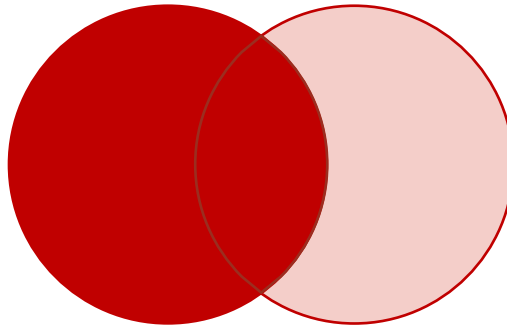
```
UPDATE table_name
SET field_name1 = value1,
    field_name2 = value2,
    ...
[WHERE expression_1
  AND expression_2
  OR expression_3]
[ORDER BY column_name [ASC|DESC] [, column_name, ...]]
[LIMIT row_count [OFFSET offset]] ;
```

III. JOIN Tables

III.1. LEFT JOIN

III.1.a. Full LEFT JOIN

Diagram

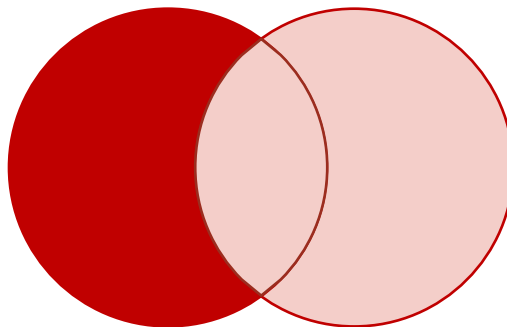


Syntax

```
SELECT *
FROM table_1
LEFT JOIN table_2 ON table_1.key = table_2.key
```

III.1.b. LEFT JOIN without intersecting B

Diagram



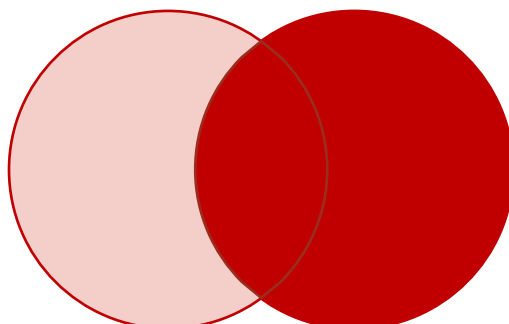
Syntax

```
SELECT *
FROM table_1
LEFT JOIN table_2 ON table_1.key = table_2.key
WHERE table_2.key IS NULL
```


III.2. RIGHT JOIN

III.2.a. Full RIGHT JOIN

Diagram

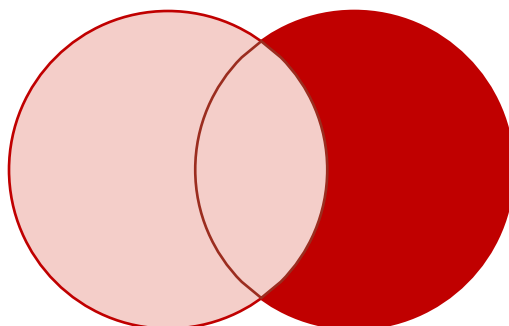


Syntax

```
SELECT *
FROM table_1
RIGHT JOIN table_2 ON table_1.key = table_2.key
```

III.2.b. RIGHT JOIN without intersecting A

Diagram

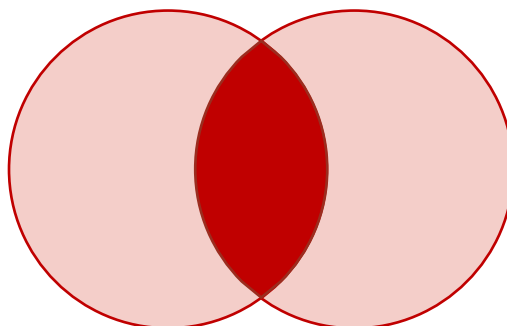


Syntax

```
SELECT *
FROM table_1
RIGHT JOIN table_2 ON table_1.key = table_2.key
WHERE table_2.key IS NULL
```

III.3. INNER JOIN

Diagram



Syntax

```
SELECT *
FROM table_1
INNER JOIN table_2 ON table_1.key = table_2.key
```

III.4. Cascade Operations

Syntax

```
CREATE TABLE child_table (
    id INT PRIMARY KEY,
    parent_id INT,
    FOREIGN KEY (parent_id)
        REFERENCES parent_table(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Other possible options:

```
ON DELETE NO ACTION
ON DELETE RESTRICT
ON DELETE SET NULL
ON DELETE CASCADE
```

Explanation

Cascade operations define what happens to rows in a child table when the referenced row in the parent table is updated or deleted.

They are used with **FOREIGN KEY** constraints to maintain referential integrity between related tables.

These actions are **triggered automatically** when a referenced record changes.

Main features

NO ACTION (default)

- This is the default behavior.
- The database checks the constraint and prevents the operation if related records exist.
- Similar behavior to **RESTRICT**.

RESTRICT

- Produces an error if a referenced record is deleted or updated.
- The operation is blocked when dependent records exist.
- Ensures that referenced data cannot be removed accidentally.

SET NULL

- Sets the foreign key value to **NULL** when the referenced record is deleted or updated.
- The child record remains, but the relationship is removed.
- The foreign key column must allow **NULL**.

CASCADE

- Automatically deletes or updates all referencing records.
- If a parent record is deleted, all related child records are deleted as well.
- Useful for maintaining consistency in dependent data.

IV. Aggregate Functions¹

IV.1. List of built-in SQL Functions

Fonction	Description
ABS()	Détermine la valeur absolue
ADDDATE(), DATE_ADD()	Ajoute des valeurs de temps (intervalles) à une date
ADDTIME()	Ajoute du temps à une heure
AVG()	Calcule la valeur moyenne
CHAR()	Détermine le caractère correspondant à l'entier
CONCAT()	Concatène une chaîne de caractères
COUNT()	Compte le nombre de lignes renvoyées
CURDATE()	Détermine la date actuelle
CURRENT_TIMESTAMP(), CURRENT_TIMESTAMP, NOW()	Détermine la date et l'heure actuelle
CURTIME()	Détermine l'heure actuelle
DATE()	Extrait la partie date d'une expression
DATEDIFF()	Soustrait deux dates
DAY(), DAYOFMONTH()	Extrait la partie jour d'une expression
EXP()	Élève à la puissance
GROUP_CONCAT()	Concatène une chaîne de caractères
HOUR()	Extrait la partie heure d'une expression
LENGTH()	Détermine la longueur d'une chaîne
LOWER()	Converti une chaîne de caractères en minuscules
MAX()	Détermine la valeur maximale
MIN()	Détermine la valeur minimale
MINUTE()	Extrait la partie minute d'une expression
MOD()	Calcule le reste d'une division entière
MONTH()	Extrait la partie mois d'une expression

¹ Given on exam cheat sheet

RAND()	Détermine une valeur aléatoire à virgule flottante
ROUND()	Arrondi une valeur numérique
SECOND()	Extrait la partie secondes d'une expression
SUM()	Calcule la somme
TIME()	Extrait la partie temps d'une expression
TRIM()	Supprime les espaces de début et de fin
UPPER()	Converti une chaîne de caractères en majuscules
YEAR()	Extrait la partie année d'une expression

Figure IV.1 – Aggregate Functions

IV.2. Detailed Function Descriptions

IV.2.a. GROUP_CONCAT()

IV.2.a.i. Syntax

```
GROUP_CONCAT([DISTINCT] expr [,expr ...]
             [ORDER BY {unsigned_integer | col_name | expr}
              [ASC | DESC] [,col_name ...]]
             [SEPARATOR str_val])
```

IV.2.a.ii. Explanation

`GROUP_CONCAT()` is an aggregate function in MySQL that concatenates values from multiple rows into a single string. It is typically used with `GROUP BY` to combine column values belonging to the same group.

IV.2.a.iii. Main features

- Combines multiple row values into one string.
- Default separator is a comma “,”.
- `DISTINCT` removes duplicate values.
- `ORDER BY` controls the order of the concatenated values.
- `SEPARATOR` allows a custom separator between values.

IV.2.b. FORMAT()¹

IV.2.b.i. Syntax

```
FORMAT(X, D [, locale])

mysql> SELECT FORMAT(12332.123456, 4);
      -> '12,332.1235'
mysql> SELECT FORMAT(12332.1,4);
      -> '12,332.1000'
mysql> SELECT FORMAT(12332.2,0);
      -> '12,332'
mysql> SELECT FORMAT(12332.2,2,'de_DE');
      -> '12.332,20'
```

IV.2.b.ii. Explanation

FORMAT() is a MySQL function that formats a number **x** into a readable number format like **#,###,###.##**. The number is rounded to **D** decimal places and the result is returned as a string. If **D** is 0, the result has no decimal point or fractional part. If **x** or **D** is **NULL**, the function returns **NULL**.

The optional locale parameter controls the decimal point, thousands separator, and grouping style. If no locale is given, MySQL uses the default locale **'en_US'**.

IV.2.b.iii. Main features

- Formats numbers with thousands separators.
- Rounds the number to the specified number of decimal places.
- Returns the result as a string.
- If **D** is 0, no decimal places are shown.
- If **x** or **D** is **NULL**, the result is **NULL**.
- The optional locale changes the number format, for example **'de_DE'** uses a dot for thousands and a comma for decimals.

¹ Not given on exam cheat sheet

V. Conditions and Operators

V.1. Numeric Comparison Operators

Operator	Condition	SQL Example
=, !=, <, <=, >, >=	Standard numerical comparisons	col_name != 4
BETWEEN ... AND ...	Value is within a given range (inclusive)	col_name BETWEEN 1.5 AND 10.5
NOT BETWEEN ... AND ...	Value is outside a given range	col_name NOT BETWEEN 1 AND 10
IN (...)	Value exists in a list of values	col_name IN (2, 4, 6)
NOT IN (...)	Value does not exist in a list	col_name NOT IN (1, 3, 5)

Table V.1 – Numeric Comparison Operators

V.2. String Comparison Operators

Operator	Condition	SQL Example
=	Case-sensitive exact match	col_name = 'abc'
!= or <>	Case-sensitive inequality	col_name != 'abcd'
LIKE	Case-insensitive pattern match	col_name LIKE 'ABC'
NOT LIKE	Case-insensitive negative pattern match	col_name NOT LIKE 'ABCD'
%	Wildcard: any sequence of 0+ characters (used with LIKE)	col_name LIKE '%AT%' (matches: AT, ATTIC, CAT, BATS)
_	Wildcard: exactly one character (used with LIKE)	col_name LIKE 'AN_' (matches: AND, but not AN)
IN (...)	String exists in a list	col_name IN ('A', 'B', 'C')
NOT IN (...)	String does not exist in a list	col_name NOT IN ('D', 'E', 'F')

Table V.2 – String Comparison Operators

V.3. Logical Operators

Operator	Meaning	SQL Example
AND	Both conditions must be true	age > 18 AND status = 'active'
OR	At least one condition is true	city = 'Paris' OR city = 'Rome'
NOT	Negates a condition	NOT (age < 18)

Table V.3 – Logical Operators

V.4. Null Operators

Operator	Meaning	SQL Example
IS NULL	Checks if value is NULL	col_name IS NULL
IS NOT NULL	Checks if value is not NULL	col_name IS NOT NULL

Table V.4 – Null Operators

V.5. Other Numeric Operators

Operator	Meaning	SQL Example
+ or -	Addition, subtraction	salary + 500
* or /	Multiplication, division	price * quantity
%	Modulo (remainder)	num % 2 = 0

Table V.5 – Other Numeric Operators

VI. References

VI.1. Bibliography

- Course material by Mr. Tobias: 01_Cours_SQL.pdf
- SQLBolt – Learn SQL with simple, interactive exercises.
<https://sqlbolt.com/lesson/introduction>
- GeeksForGeeks – “Normal Forms in DBMS” – updated on 20.11.2025
<https://www.geeksforgeeks.org/dbms/normal-forms-in-dbms/>
(accessed on 10.12.2025)
- MySQL – “14.19.1 Aggregate Function Descriptions” – version 9.7
https://dev.mysql.com/doc/refman/8.4/en/aggregate-functions.html#function_group_concat
(accessed on 01.04.2026)
- MySQL – “14.8 String Functions and Operators” – version 9.7
https://dev.mysql.com/doc/refman/9.7/en/string-functions.html#function_format
(accessed on 27.04.2026)

VI.2. Sources of Illustrations

- Cover page illustration (page 1):
Spark – “Choosing the right database for an app”
<https://www.okoone.com/spark/product-design-research/choosing-the-right-database-for-an-app/>
(accessed on 10.12.2025)